Coding for Designers

UBC School of Architecture and Landscape Architecture
2026 Summer | ARCH 577c / DES450i

Online
Mondays & Wednesdays
6:30 PM - 8:00 PM
May 11 - August 12

Instructor	GTAs	
Thomas Gaudin	**Lydia Elbatarny**	**Karl Lam**
tgaudin@sala.ubc.ca	lyd18le7@mail.ubc.ca	karllamwn@gmail.com
Office Hours:	Office Hours:	Office Hours:
By Email Request	Mondays - TBD	Mondays - TBD

Dean Vietor- New Yorker - 1993

Overview

Coding for Designers will equip students with fundamental programming skills, tailored to the architectural research and practice. In today's evolving field, programming proficiency is increasingly useful and desired. It empowers practitioners to automate scheduling and drawing tasks, create intricate geometries, perform sophisticated urban and programmatic analyses, and customize design tools to meet specific needs.

In particular, this course will focus on two topics:
- Geospatial Analysis and Visualization
- Geometry Generation

Programming does not need to be unnecessarily difficult. **In the first portion of the course students will learn the basics of programming** in interactive Python notebooks, where feedback is instantaneous. Through live feedback and iteration, students can foster an intuition for the mechanics of coding. In particular, an emphasis will be placed on programming in the object oriented paradigm, which will increase students' comprehension of geometry and spatial analysis libraries used in later portions of the course.

Following the basics, in the **second portion of the course, students will utilize Python in Grasshopper to create 3D geometry. A basic overview of Grasshopper will be provided but the focus will remain on written code in Python.**

The third portion of the course will give a brief overview of data-wrangling and spatial analysis utilizing industry standard Python libraries. Students will be taught how to import, and structure data, and how to visualize their analyses in Python interactive notebooks.

Through this course, students will enhance their ability to undertake intricate analyses for studio projects, generate computational geometry, and efficiently handle the abundance of data accessible to architects. Even if they choose not to further develop their coding skills beyond the course topics, they will acquire a foundational understanding of programming that will remain applicable despite technological advancements in the field of computational design.

Class Structure

The course content will primarily be taught in a lecture format. Lecture notes will be provided as *interactive python notebook* files (.ipynb) and will be available for download on Canvas before a given lecture.

Readings will be assigned for each class, and lectures will expand upon the readings. ***You must read the assigned readings before the lecture.*** Reading links are found at the beginning of each lecture's notebook file.

Students are encouraged to work in pairs or groups for the entirety of the course. It's much easier to learn to code with someone to talk to!

This course will be delivered online, via Zoom. All lectures will be recorded and links to lecture recordings will be uploaded to Canvas. Realtime attendance is greatly encouraged.

Lecture and Office Hours Zoom Link

https://ubc.zoom.us/j/66135841413?pwd=ODseV2xqlzV1eOv6QWKoLB3vM8efu7.1
Meeting ID: 661 3584 1413
Passcode: 866352

Support

TAs will be available to help during scheduled, online office hours and via e-mail. Office hours topics of review will be posted in the ***Discussions*** section of Canvas. Students are encouraged to comment via Canvas to request topics for review. Students can also attend Lydia's office hours for help outside of the requested topics or for assistance in their assignments.

Students are encouraged to google everything! Python is extremely well documented and most answers can be found within a few minutes of searching. The lecture notes will also contain links to online resources which provide a good starting point for further research.

Course AI Policy

Students are permitted to use artificial intelligence tools, including generative AI, to gather information, review concepts or to help produce assignments. However, students are ultimately accountable for the work they submit, and any content generated or supported by an artificial intelligence must be submitted with AI assistant transcripts.
https://academicintegrity.ubc.ca/genai/

Required Readings

We will be referencing portions of two books:

Downey, Allen B. *Think Python: How to Think Like a Computer Scientist. 3rd Edition*. O'Reilly Media, 2024. (Link)

Matthes, Eric. *Python Crash Course, 3rd Edition: A Hands-On, Project-Based Introduction to Programming.* No Starch Press. (Link)

Both books can be accessed online, for free, though your UBC library account. Log in though this link and then access the required readings though the links above.

In addition to the two books, we will be referencing online resources. ***All required readings must be done before the class. All required readings are posted at the beginning of each lecture file.***

Software Requirements

Rhino 8 - Mac or Windows
Rhino 7 is not compatible with the content of this course. A free 90-day trial and/or purchased student license for Rhino 8 can be found here.

Python 3.14.x
Download the current version of Python here.

Visual Studio Code - Latest Version
Download the current version of VS Code here.

Chrome Remote Desktop (for Instructor and TA Remote Help)
Setup Remote Desktop here.

Attendance

Attendance will not be recorded, but students are encouraged to attend every class.

Academic Accommodations

Any student with a documented disability needing academic accommodation should speak to the instructor during the first two weeks of class. Further help can be found via UBC's Academic Accommodations for Students with Disabilities.

Evaluation

A final mark for the course will be provided based on an average of 3 assignments and one mid-term exam broken down as such:

Assignment 01 - Basic Python = 20%
Assignment 02 - Python Grasshopper = 30%
Assignment 03 - Geospatial Analysis = 30%
Code Comprehension Interview = 20%

The first assignment will be graded automatically where marks are given upon successful unit tests.

Assignments 02 and 03 will be evaluated and graded on these overall criteria: complexity of ideas and quality of production, code style and legibility, application of course content, and demonstration of programming abilities.

The code comprehension interview is a one-on-one informal chat with Thomas where you will have to talk through topics taught within the previous 4 lectures. In lieu of a midterm, and given the permissive AI policy of the course, comprehension will be tested verbally.

Late Work

You are expected to meet all due dates. Extensions will be given only when eligibility for doing so, as outlined by UBC's Academic Concessions* policy is met. At any point in the term, students with unsatisfactory progress may receive a letter of concern with recommendations for improved performance. Late work will be subject to a 5%-per-day penalty. Late work will no longer be accepted after the due date of the next exercise.

* https://students.ubc.ca/enrolment/academic-learning-resources/academic-concessions

Program Prerequisites

Students are expected to have at least a working knowledge of Rhino3d. If you have never used Rhino3D before, you are encouraged to learn the basics by mid-June. Excellent tutorials for Rhino can be found on Mcneel's tutorials page.

Grasshopper is not a prerequisite for this course, but you are encouraged to spend a few hours of your mid-term break playing around with demo files that will be provided.

Land Acknowledgement

This course acknowledges UBC's occupation of the traditional, ancestral, unceded territory of the Musqueam people. Find out more about this land acknowledgement here:
https://students.ubc.ca/ubclife/whatland-acknowledgement

UBC Values and Policies

UBC provides resources to support student learning and to maintain healthy lifestyles but recognizes that sometimes crises arise and so there are additional resources to access including those for survivors of sexual violence. UBC values respect for the person and ideas of all members of the academic community. Harassment and discrimination are not tolerated nor is suppression of academic freedom. UBC provides appropriate accommodation for students with disabilities and for religious and cultural observances. UBC values academic honesty and students are expected to acknowledge the ideas generated by others and to uphold the highest academic standards in all of their actions. Details of the policies and how to access support are available here: https://senate.ubc.ca/policies-resources support-student-success.

Academic Integrity

The academic enterprise is founded on honesty, civility, and integrity. As members of this enterprise, all students are expected to know, understand, and follow the codes of conduct regarding academic integrity. At the most basic level, this means submitting only original work done by you and acknowledging all sources of information or ideas and attributing them to others as required. This also means you should not cheat, copy, or mislead others about what is your work. Violations of academic integrity (i.e.,misconduct) lead to the breakdown of the academic enterprise, and therefore serious consequences arise and harsh sanctions are imposed. For example, incidences of plagiarism or cheating may result in a mark of zero on the assignment or exam and more serious consequences may apply if the matter is referred to the President's Advisory Committee on Student Discipline. Careful records are kept in order to monitor and prevent recurrences. A more detailed description of academic integrity, including the University's policies and procedures, may be found in the Academic Calendar here: http://calendar.ubc.ca/vancouver/index.cfm?tree=3,54,111,0

Respectful Environment

The University of British Columbia envisions a climate in which students, faculty and staff are provided with the best possible conditions for learning, researching and working, including an environment that is dedicated to excellence, equity and mutual respect. The University of British Columbia strives to realize this vision by establishing employment and educational practices that respect the dignity of individuals and make it possible for everyone to live, work, and study in a positive and supportive environment, free from harmful behaviours such as bullying and harassment. More here: https://hr.ubc.ca/working-ubc/respectful-environment

Schedule

Course Portion	Day	Date	Topic
Programming Fundamentals	Mon	May 11	Introduction / Workspace Setup
	Wed	May 13	Using AI
	Mon	May 18	Victoria Day - No Class
	Wed	May 20	Immutable Types, Functions, Operators
	Mon	May 25	Flow Control and Iteration (While loops)
	Wed	May 27	Lists and Iteration (For Loops)
	Mon	June 1	Dictionaries / Overflow
	Wed	June 3	Objects / Mutability
	Mon	June 8	Work / Help Session
	Wed	June 10	Building a Pokemon Battling Game 01
	Mon	June 15	Building a Pokemon Battling Game 02
Computational Geometry	Wed	June 17	Grasshopper Basics 01
			Mid-Term Break - No Classes
	Mon	July 6	Grasshopper Basics 02
	Wed	July 8	Grasshopper Python 1
	Mon	July 13	Grasshopper Python 2
	Wed	July 15	Grasshopper Python 3
	Mon	July 20	Work / Help Session
Geospatial Analysis	Wed	July 22	Structured Data and GeoPandas
	Mon	July 27	Geospatial Analysis 01 - Data Wrangling
	Wed	July 29	Geospatial Analysis 02 - Spatial Joins
	Mon	Aug 3	BC Day - No Class
	Wed	Aug 5	Geospatial Analysis 03 - Cleanup and Export
	Mon	Aug 10	Work / Help Session
	Wed	Aug 12	Work / Help Session